

Übungsblatt 2 – Bericht

3D-NPC mit KI-gesteuertem Verhalten

Jonathan Stengl

Konzept

In das Labyrinth wurde ein NPC integriert, dessen Aufgabe es ist, den Spieler zu finden und frontal zu berühren. Sobald dies geschieht, wird das Spiel gestoppt und der Spieler hat verloren. Der NPC soll den Spieler also aktiv daran hindern, das Labyrinth zu verlassen. Die Entscheidung, wohin sich der NPC als Nächstes bewegt und mit welcher Geschwindigkeit, trifft ein integriertes Large Language Model (LLM). Das LLM erhält keine exakte Position des Spielers, sondern lediglich einen vom Spiel berechneten Suchradius (Zentrum + Größe), innerhalb dessen sich der Spieler befindet. Durch bestimmte Verhaltensweisen beeinflussen sowohl der Spieler als auch der NPC die Größe dieses Suchradius:

- **Beispiel 1:** Wenn der Spieler rennt, wird der Radius kleiner, sodass das LLM eine präzisere Eingrenzung erhält – der NPC wird dadurch aufmerksamer. So kann der Spieler beim NPC ein gewisses Verhalten provozieren, denn das LLM weiß jetzt genauer, wo sich der Spieler befindet und könnte zum Beispiel anfangen zu rennen.
- **Beispiel 2:** Wenn der NPC selbst schnell rennt, führt dies zu einem größeren Radius – eine Art "Geräuschsimulation", die das LLM berücksichtigt.
- **Beispiel 3:** Sinkt die Distanz zwischen Spieler und NPC, wird der Radius ebenfalls kleiner, um ein realistisches Geräuschempfinden zu simulieren.

Das LLM muss in jeder Situation abwägen: Welche Bewegungsgeschwindigkeit ist sinnvoll? Wohin sollte der NPC steuern? Dadurch ergibt sich ein glaubwürdiges, dynamisches Verhalten.

Animationen und Bewegungsverhalten

Der NPC kann sich mit Geschwindigkeiten zwischen 0 und 5 m/s fortbewegen. Die Bewegung wird durch passende Animationen ergänzt:

- **0 m/s:** Idle-Animation
- **0,01 – 1,5 m/s:** Geh-Animation
- **ab 1,5 m/s:** Renn-Animation

Kriechen

Wird der NPC von hinten vom Spieler berührt, löst dies eine Hinfall-Animation aus. Anschließend wechselt der NPC in eine Kriech-Animation mit fixer Geschwindigkeit (0,5 m/s). Die Kriechdauer wird vom Spiel zufällig (innerhalb festgelegter Grenzen) bestimmt. Während dieser Phase kann das LLM nur das Ziel bestimmen, aber nicht die Geschwindigkeit beeinflussen. Nach Ablauf der Kriechzeit erfolgt eine Aufsteh-Animation, und der NPC kehrt in seinen Normalzustand zurück.

Komponenten des NPC

Der NPC wurde als humanoides Objekt gestaltet und mit mehreren funktionalen Komponenten ausgestattet, um sich realistisch in der Spielumgebung bewegen und mit dieser interagieren zu können. Die Folgenden Komponenten werden genutzt:

Animator

Der Animator steuert die Übergänge zwischen verschiedenen Bewegungszuständen des NPC und sorgt so für glaubwürdige und flüssige Animationen. Die Übergänge basieren auf die zuvor im Konzept definierten Bedingungen. Im Animator-Controller sind sämtliche Zustandswechsel und ihre Voraussetzungen modelliert (vgl.: Abbildung 1: Animator)

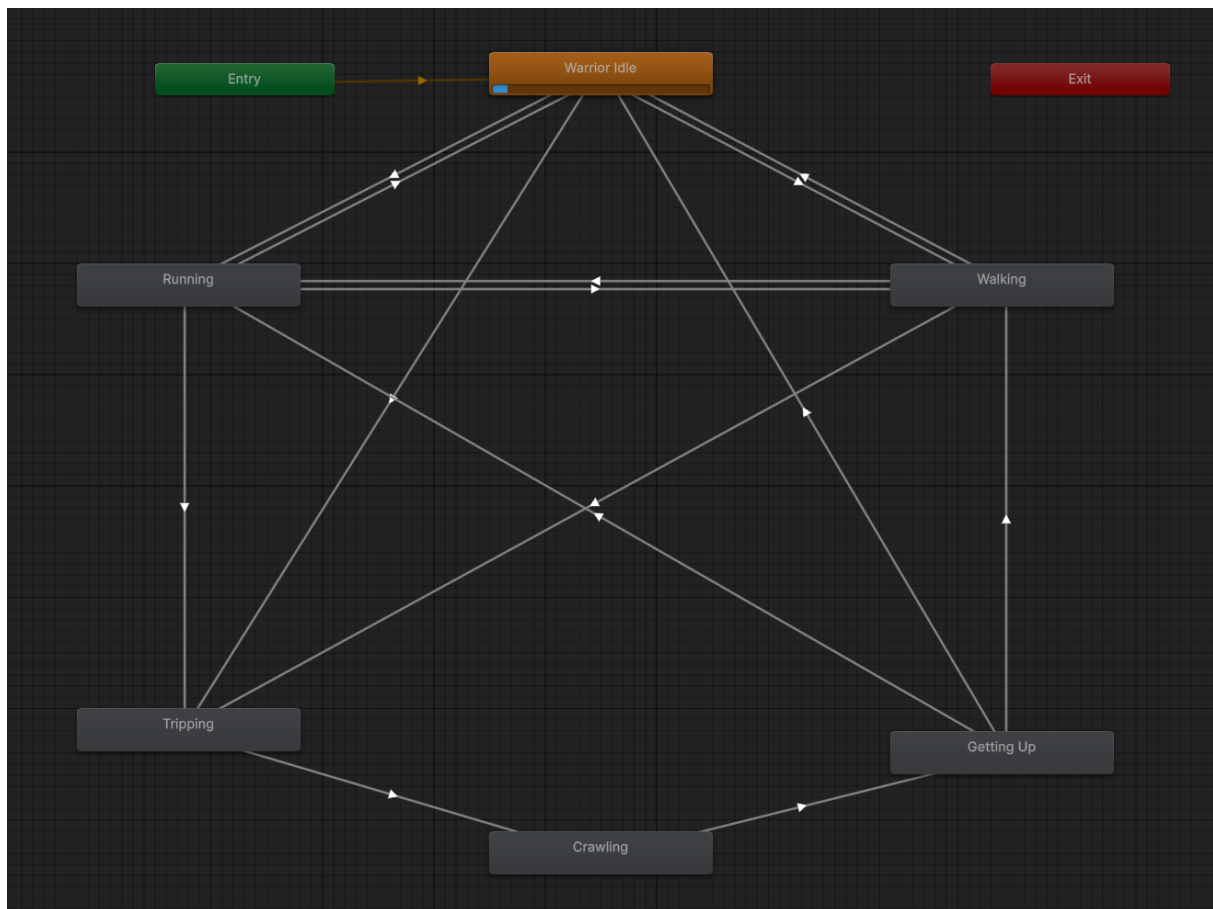


Abbildung 1: Animator

Die Steuerung erfolgt über folgende Parameter:

- **speed (float):** Steuert die Übergänge zwischen Stehen, Gehen und Rennen.
- **knock (Trigger):** Aktiviert die Hinfall-Animation, z. B. bei einer Kollision mit dem Spieler.
- **standUp (Trigger):** Startet die Aufsteh-Animation nach dem Kriechen.

Nav Mesh Agent

Der Nav Mesh Agent ist für die autonome Navigation des NPC verantwortlich. Sobald das LLM ein Ziel vorgibt, übernimmt der Agent die Wegfindung innerhalb des Labyrinths. Dank des zuvor gebackenen Nav Mesh Surface kann der Agent den kürzesten gangbaren Weg ermitteln und Hindernisse wie Wände umgehen. Dadurch ist keine eigenständige Pfadberechnung durch das KI-Modell erforderlich.

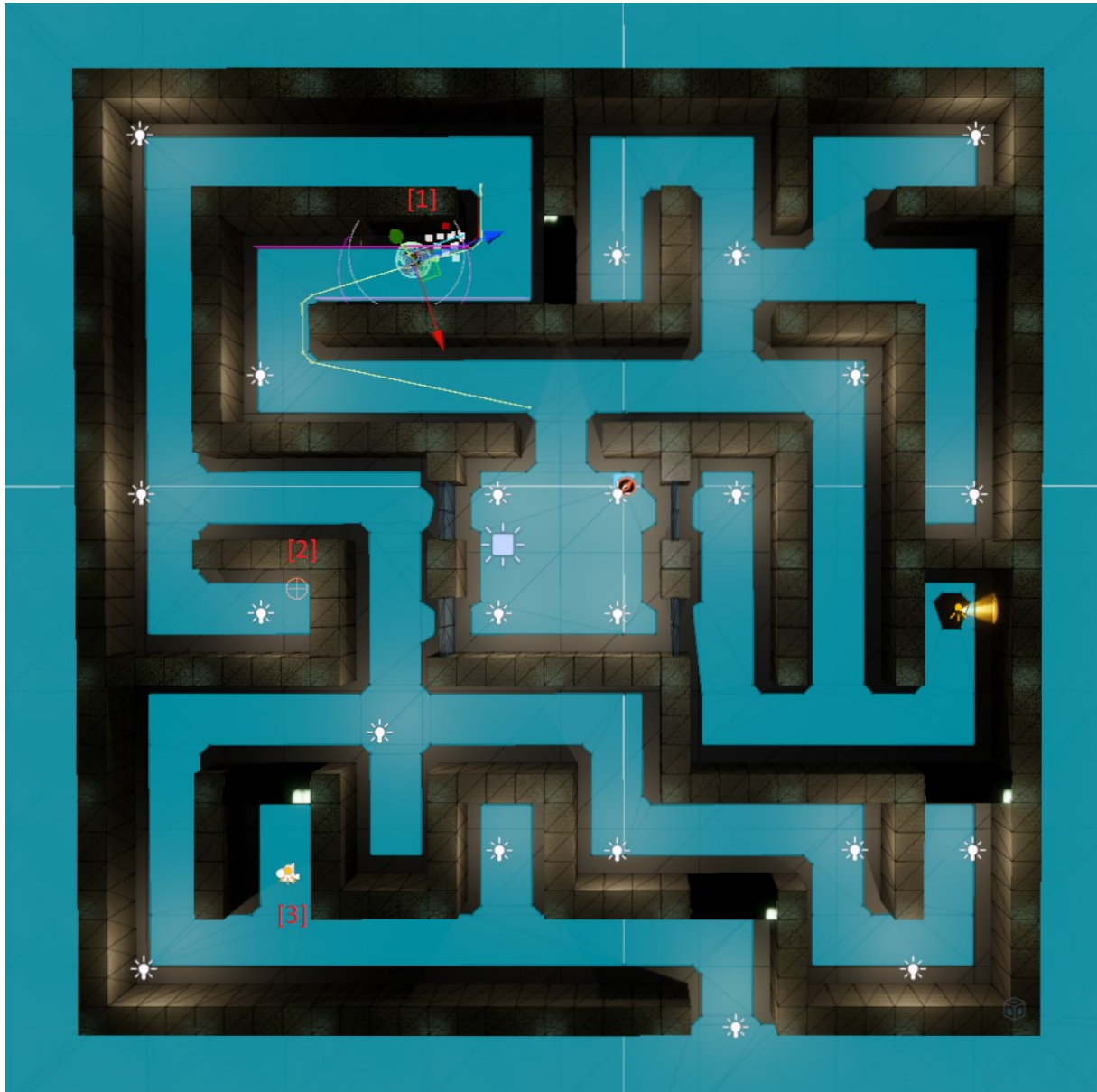


Abbildung 2: Nav Mesh Agent

Legende zur Abbildung:

- **[1]** Suchender NPC
- **[2]** Zielposition, vom LLM vorgeschlagen
- **[3]** Spieler
- **Cyan-Fläche:** Begehbare Bereiche des Nav Mesh

Capsule Collider

Zur physikalischen Interaktion ist der NPC mit zwei Capsule Collidern ausgestattet:

- **Der erste Collider** fungiert als Trigger und erkennt eine Berührung mit dem Spieler. Diese kann eine Reaktion wie Hinfallen oder das Beenden des Spiels auslösen.
- **Der zweite Collider** dient als physikalische Kollisionseinheit, um sicherzustellen, dass der NPC eine Hitbox besitzt und nicht durchquert werden kann.

Interaktion zwischen NPC und LLM

Die Verbindung zwischen NPC und KI erfolgt über zwei zentrale Skript-Komponenten: „NpcLlmDriver“ und „LlmBrain“. Gemeinsam ermöglichen sie ein dynamisches, situationsabhängiges Verhalten des NPC basierend auf den Entscheidungen eines Large Language Models (LLM).

NpcLlmDriver

Das Skript NpcLlmDriver übernimmt die Steuerung des NPCs auf Basis der Rückmeldungen des LLM und koordiniert dabei sowohl Bewegungslogik als auch Kontextanalyse. Es wird direkt auf das NPC-Objekt angewendet erfüllt folgende zentrale Aufgaben:

Dynamische Radiusberechnung

In jedem Frame wird der Suchradius rund um den Spieler neu berechnet. Abhängig von der Distanz und der Bewegungsgeschwindigkeit des Spielers und des NPCs (über die öffentlichen Parameter *playerWalkFac*, *playerRunFac*, *npcWalkFac* und *npcRunFac*) wird der Radius entsprechend vergrößert oder verkleinert. So beeinflusst das Verhalten beider Figuren die Informationslage der KI aktiv.

Anfrageintervall-Regelung

Die Frequenz, mit der das LLM neue Zielpunkte berechnet, hängt von der Distanz zum aktuellen Ziel ab. Je weiter das Ziel entfernt ist, desto seltener wird eine neue Anfrage ausgelöst. Über die Parameter *minQueryDelay* und *maxQueryDelay* können die minimalen bzw. maximalen Abstände zwischen zwei Anfragen festgelegt werden. Dies reduziert unnötige Anfragen und erzeugt ein zielstrebigeres, glaubwürdiges Verhalten.

Koordination des Sturz- und Kriechverhaltens

Erkennt das System eine Berührung von hinten (z. B. durch den Spieler), wird die NPC-Bewegung gestoppt und eine Hinfall-Animation ausgelöst. Anschließend kriecht der NPC für eine zufällig gewählte Zeitspanne (*minCrawlTime* bis *maxCrawlTime*), bevor er wieder aufsteht.

Nav Mesh-Verknüpfung

Das vom LLM zurückgelieferte Ziel und die Geschwindigkeit werden direkt an den Nav Mesh Agent übergeben, der für die konkrete Navigation innerhalb des Labyrinths zuständig ist.

Datenbereitstellung für das LLM

NpcLlmDriver erzeugt ein JSON-Objekt mit allen relevanten Informationen über den aktuellen Zustand des NPCs und seiner Umgebung. Dieses wird zur Entscheidungsfindung an das LLM gesendet. Das Format ist wie folgt:

```

{
  "radius": 13.193,           // der Radius des Suchbereichs
  "radius_center": [9.215, 3.197], // das Zentrum des Suchbereichs
  "npcPos": [18.941, -1.931], // die aktuelle Position des NPC
  "history": [                // Informationen über vergangene Zustände und Entscheidungen -> das Gedächtnis
    {
      "timestamp": 0.096,      // Zeitpunkt der Handlung
      "speed": 0.0,           // Geschwindigkeit des NPC
      "target": [0.000, 1.013], // gewähltes Ziel
      "radius": 11.344,        // der damalige Radius
      "radius_center": [0.000, 0.000], // das damalige Zentrum des Radius
      "npcPos": [14.080, -27.660] // die damalige Position des NPC
    },
    {
      "timestamp": 5.979,
      "speed": 2.5,
      "target": [16.000, -20.000],
      "radius": 5.460,
      "radius_center": [16.000, -16.000],
      "npcPos": [14.084, -27.669]
    },
    //....
  ]
}

```

Auf Basis dieser bereitgestellten Informationen ist das LLM in der Lage, sich situationsabhängig und systematisch dem Spieler zu nähern – ohne diesen direkt „sehen“ oder gezielt anvisieren zu müssen. Es erhält lediglich einen Suchradius mit Zentrum und Größe, der grob den Aufenthaltsort des Spielers umreißt. Dadurch wird vermieden, dass das Modell durch Wände „hindurchsehen“ kann. Die Informationslage ist so gewählt, dass eine nachvollziehbare, indirekte Suche möglich ist, die dennoch zielgerichtet wirkt.

Über den Parameter *historyDepth* lässt sich zudem festlegen, wie groß das „Gedächtnis“ des LLMs ist, also wie viele vergangene Züge es in seine Entscheidungen einbeziehen darf.

LlmBrain

Die Kommunikation mit dem KI-Modell erfolgt nicht direkt über den NpcLlmDriver, sondern über die spezialisierte Hilfsklasse LlmBrain. Diese ist verantwortlich für die API-Kommunikation mit dem LLM und wird beim Spielstart durch NpcLlmDriver initialisiert.

Verwendete Konfiguration in diesem Projekt:

- **Modell:** gpt-4o-mini (OpenAI)
- **Temperatur (Kreativitätsfaktor):** 0.7
- **Maximale Tokenanzahl:** 1000
- **Kommunikation:** HTTP POST-Requests an die OpenAI-API

Zur Initialisierung werden zwei TextAsset-Dateien benötigt:

- Eine Datei mit dem API-Schlüssel für den Zugriff auf das LLM.
- Eine Datei mit der Systemanweisung (Prompt), in der das Verhalten des NPCs definiert ist.

Die Systemanweisung klärt das Modell über seine Aufgabe auf: Es soll den Spieler möglichst schnell finden, dabei neue Areale erkunden und sich überlegen, wann welche Bewegungsgeschwindigkeit sinnvoll ist. Außerdem wird darauf hingewiesen, dass es den dynamischen Radius beachten muss und wie es die mitgegebenen Parameter – insbesondere Position, Radiuszentrum und Verlaufshistorie – interpretieren soll. Die Einträge vergangener Entscheidungen sollen genutzt werden, um Dopplungen zu vermeiden und strategisch neue Bereiche abzudecken. Zusätzlich wird klar definiert, welches JSON-Format die Antwort haben muss – bestehend aus einem Zielpunkt und einer Geschwindigkeit:

```
{  
  "speed": <float>,    // 0–5 m/s  
  "target": [<x>, <z>] // Koordinaten innerhalb des Suchbereichs  
}
```

Anhang

Nachfolgend ist in Abbildung 3 der Ablauf der NPC-Verhaltensroutine grafisch dargestellt. Während dieses Prozesses kann grundsätzlich jederzeit ein Kontakt mit dem Spieler erfolgen, wie in Abbildung 4 dargestellt. Eine Ausnahme bildet der Zustand, in dem der NPC bereits gestürzt ist oder sich noch im Aufstehvorgang befindet – in diesem Fall bleibt eine Berührung durch den Spieler wirkungslos und hat keine weiteren Auswirkungen auf den Spielverlauf.

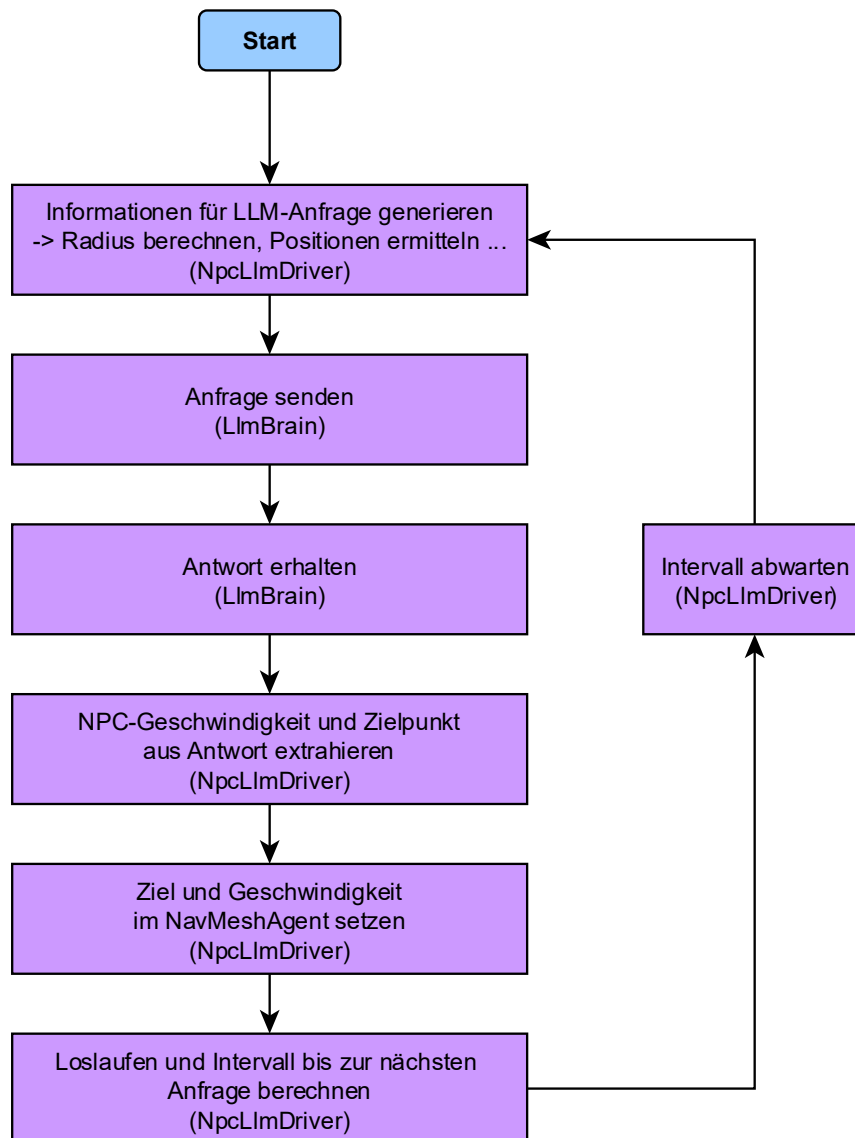


Abbildung 3: Routineablauf

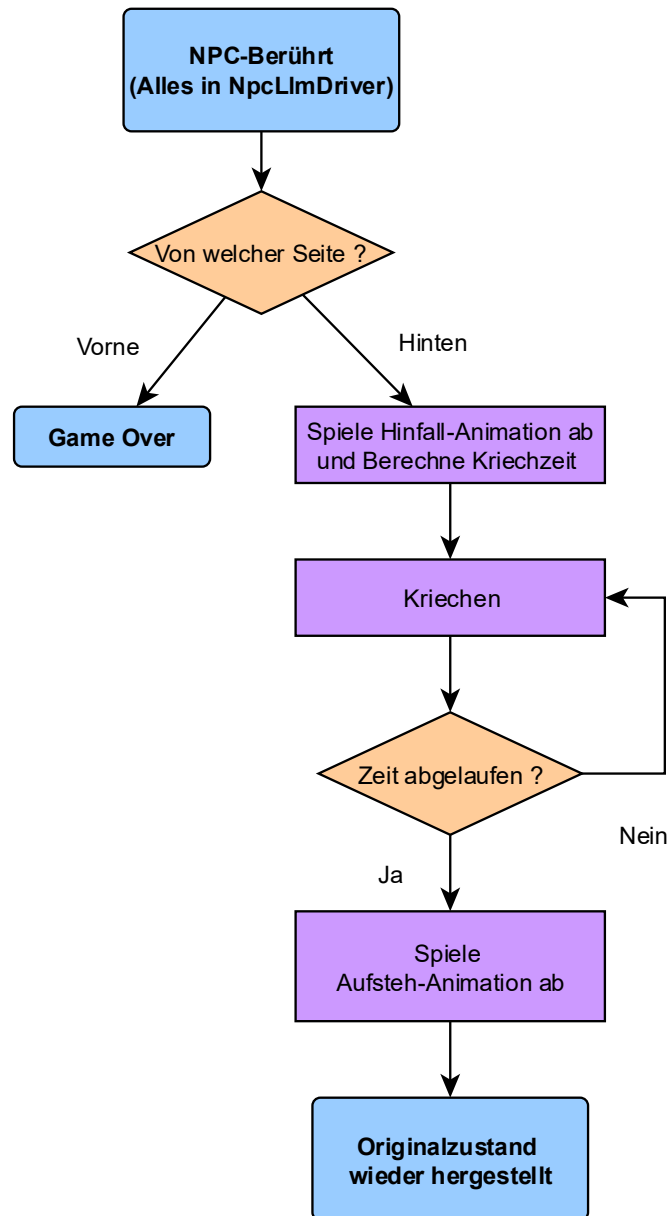


Abbildung 4: NPC-Kontakt

Quellen (05.07.2025)

- Unity: <https://unity.com/de>
- Mixamo für NPC-Model und Animationen: <https://www.mixamo.com/#/>
- NPC-und Animationsgrundlagen: <https://www.youtube.com/watch?v=0QA2O7juuWQ>
- NavMesh-Grundlagen: <https://www.youtube.com/watch?v=SMWxCpLvrcc>
- LLM von OpenAI: <https://openai.com/>